

MIZAN

ميزان

Product Requirements Document

Internal Business Management & Inventory System — Version 1.1

Document version	1.1 — Implemented schema baseline
Status	Database schema, models and seed data implemented and verified
Date	16 September 2026
Product owner	Ayob Milad
Technology	Laravel 13.31 · PHP 8.4 · MySQL 8.4 · Blade · Tailwind 4 · Alpine.js 3
Supersedes	PRD V1.0 (13 Sep 2026)

Revision history

Version	Date	Summary
1.0	13 Sep 2026	Consolidated requirements, use cases and ERD decisions. Resolved eight conflicts between the source papers. Design baseline.
1.1	16 Sep 2026	Every decision taken while implementing the database: authentication hardening, multi-unit integrity, money rounding policy, cash-only returns enforced in the database, immutable history, atomic document numbering. Section 8 is now generated directly from the live schema. Full change list in Appendix D.

Contents

1. Purpose and Document Authority

2. System Overview and Scope

3. Actors and Responsibilities

4. Global Business Rules

5. Resolved Design Conflicts (C1–C8)

6. Supplementary Design Decisions

7. Functional Requirements

8. Data Model

Integrity architecture · Database-enforced invariants · Service-enforced rules · Schema reference (31 tables)

9. Technical Architecture and Implementation Status

10. Security Requirements

11. Out of Scope — V2 Roadmap

Appendix A — Permission Catalogue

Appendix B — Role / Permission Matrix

Appendix C — Decision Register

Appendix D — Change Log V1.0 → V1.1

Appendix E — Open Items

1. Purpose and Document Authority

This document is the single authoritative specification for **Mizan V1**. Version 1.1 records the decisions taken while the database was implemented and verified against MySQL. Where it differs from V1.0 or any earlier paper, **this document governs**.

Schema accuracy. The schema reference in section 8.5 is generated from the running database (`information_schema`), not written by hand. Column types, keys and constraints in this document are therefore exactly what is deployed.

1.1 Design philosophy

Property	What it demands of the implementation
Transactional integrity	Every business operation that touches more than one row succeeds completely or not at all.
Inventory traceability	No quantity ever changes without a stock movement recording the cause.
Concurrency safety	Correctness is guaranteed by database locks and constraints, never by application checks alone.
Auditability	Sensitive actions answer: who, what, when, where, which record, what changed, why.
Accountability	Every operation carries the identity of the employee who performed it.
Historical integrity	Completed transactions are immutable. Corrections are new operations, never edits.
Defence in depth V1.1	A business rule that the database can express is enforced there as well as in code, so a programming error cannot corrupt financial or inventory data.

2. System Overview and Scope

Mizan is an internal business management and inventory system for a retail business with warehouses in several cities. V1 serves one company and is not multi-tenant.

2.1 In scope

Domain	Capabilities
Identity	Authentication, invitation-based onboarding, roles, permissions
Master data	Products, product units, categories, brands, warehouses, customers, suppliers
Inventory	Per-warehouse balances, stock movements, adjustments, inter-warehouse transfers
Purchasing	Purchase orders, confirmation, cancellation, goods receiving
Selling	Sales sessions, invoices, multi-warehouse selling, walk-in customers, invoice-level discount, tax
Returns	Multi-item returns against cash sales, cash refunds, reconciliation impact
Oversight	Reports, audit log, system settings

2.2 Out of scope for V1

Delivery representatives; customer accounts and portal; order and delivery tracking; real-time status; customer chat; WebSocket infrastructure; online payment gateways; product variants; multi-tenancy; partial goods receiving; line-level discounts; returns of card or transfer sales; returning a different unit than the one sold.

3. Actors and Responsibilities

Actor	Can do	Cannot do
Admin 51 permissions	Everything, including employees, roles, permissions, settings and the audit log.	—
Manager 31 permissions	Products, categories, brands, suppliers, customers, warehouses. Create and confirm purchase orders. View inventory, movements, all sessions, sales, returns and all reports.	Users, roles, settings, audit log. Create sales or returns. Adjust or transfer stock. Receive goods.
Sales Employee 11 permissions	Start and close own session, sell from any warehouse, create and edit customers, view own sales and returns, execute returns and cash refunds, view products and stock.	View other employees' sales. Modify completed sales. Deactivate customers. Inventory operations.

Warehouse Employee 9 permissions	View inventory, movements, products, suppliers and warehouses. Receive purchases, adjust stock, transfer stock.	Create sales. Manage users, roles or permissions.
Customer	A business entity associated with sales.	No login, password, role or dashboard in V1.

Separation of duties. The Manager reviews but does not execute. A business that needs one person to do both creates a new named role holding both permission sets: an explicit, audited decision rather than a weakening of the defaults.

4. Global Business Rules

ID	Rule
BR-GLOBAL-01	Users must be authenticated before any protected operation.
BR-GLOBAL-02	Every protected operation is authorised on the backend against the user's role permissions.
BR-GLOBAL-03	The backend calculates and validates prices, totals, taxes, discounts, stock, warehouse access, the active session, refund amounts and permissions. None of these is trusted from the client.
BR-GLOBAL-04	Records with history are never hard-deleted; a <code>status</code> column deactivates them. Soft deletes are not used. Foreign keys default to <code>RESTRICT</code> .
BR-GLOBAL-05	Operations changing several related records run in one database transaction.
BR-GLOBAL-06	Sensitive and inventory operations are recorded in the audit log.
BR-GLOBAL-07	Stock can never become negative by any operation, including adjustment. Enforced by <code>CHECK</code> and by application validation.
BR-GLOBAL-08	Rows in a read-check-write sequence are locked with <code>SELECT ... FOR UPDATE</code> until commit.
BR-GLOBAL-09 V1.1	Money is rounded half away from zero to 3 decimals. PHP <code>bcround(..., 3, HalfAwayFromZero)</code> and MySQL <code>ROUND(..., 3)</code> were verified to agree. Base-unit quantities are truncated to 3 decimals: PHP <code>bcmul</code> and MySQL <code>TRUNCATE</code> agree. Floating point is never used for money or quantities.
BR-GLOBAL-10 V1.1	Completed documents (sales, sale lines, returns, return lines, refunds, receipts, adjustments, transfers, stock movements, audit entries) cannot be updated or deleted through the application. The only exception is the returned-quantity counter on a sale line.
BR-GLOBAL-11 V1.1	Operations that a double click could duplicate carry an idempotency key under a unique constraint: sales, returns, stock adjustments and stock transfers. Purchase receipts are protected by one-receipt-per-order.

5. Resolved Design Conflicts

These eight decisions from V1.0 stand. V1.1 adds how each is enforced. **CANCELLED** rules must not be reintroduced.

#	Conflict	Decision	Enforcement in V1.1
C1	Is a sales session tied to one warehouse?	No. The session belongs to the employee; each sale records its own warehouse. CANCELLED BR-SESSION-05/06/07/08.	<code>sales_sessions</code> has no warehouse column; <code>sales.warehouse_id</code> is mandatory.
C2	GraphQL + Vue, or Blade?	Blade + Tailwind + Alpine.js. No GraphQL, no Vue in V1.	Implemented (Laravel 13, Tailwind 4, Alpine 3).
C3	Purchase tables	Order and receipt are separate: four tables.	One receipt per order by unique index. Receipt lines reference order lines.
C4	Payment methods and returns	cash / card / transfer. Returns only on cash sales.	Enforced in the database by a composite foreign key and a CHECK (\$8.2).
C5	Opening cash float	Entered by the employee at session start.	<code>expected = float + cash sales - cash refunds</code> ; closing requires <code>difference = actual - expected</code> .
C6	Where does returned stock go?	Always to the original sale's warehouse.	No warehouse column on returns; it is derived from the sale.
C7	Return structure	Multi-item return, exactly one refund per return.	Unique <code>refunds.return_id</code> ; refund amount must equal the return total.
C8	Roles per user	One role per user. CANCELLED <code>role_user</code> .	<code>users.role_id</code> ; role in use cannot be deleted (RESTRICT).

6. Supplementary Design Decisions

6.1 Multi-unit products

Goods are bought wholesale and sold retail. Each product has units (piece, dozen, carton) with a conversion factor and explicit per-unit prices; a carton is not priced by multiplying the piece price.

Governing rule. Stock and stock movements are always in the **base unit**. Every transaction line freezes `unit_factor` and stores `base_quantity = TRUNCATE(quantity × unit_factor, 3)`.

- Exactly one base unit per product, whose factor must be 1 (generated column + unique index + CHECK).
- V1.1** A line's unit must belong to the line's product: composite foreign key (`product_unit_id`, `product_id`) → `product_units(id, product_id)` on purchase, transfer and sale lines.
- V1.1** Adjustment lines hold balances only and are always in base units; the unit a clerk counted in is converted by the screen before saving.
- V1.1** Returns are expressed in the unit that was sold.

```
2 cartons sold, factor 24      base_quantity = 48      stock decreases by 48 pieces
profit = line_total - (base_quantity × unit_cost per base unit)
```

6.2 Discounts, tax and line amounts

- Discount is a percentage on the whole invoice, default 0. Percentage and amount are both stored.
- V1.1** `tax_rate` is frozen on each sale, so later setting changes never alter the meaning of old invoices. Tax is applied **after** the discount.
- The discount is distributed across lines. Each line stores its `discount_share`, its `line_total` and its `net_unit_price`.
- Refunds use the net (post-discount) price: a piece sold at 1000 with a 20% invoice discount refunds 800.

```
discount_amount = ROUND(subtotal × discount_percent / 100, 3)
tax_amount      = ROUND((subtotal - discount_amount) × tax_rate / 100, 3)
total           = subtotal - discount_amount + tax_amount
line_total      = ROUND(quantity × unit_price, 3) - discount_share
net_unit_price  = ROUND(line_total / quantity, 3)
```

All five formulas are CHECK constraints in the database.

6.3 Precision

Concern	Decision
Money	<code>decimal(15,3)</code> (Libyan dinar has three decimal places)
Quantities	<code>decimal(12,3)</code> ; conversion factors <code>decimal(12,4)</code> ; rates <code>decimal(5,2)</code>
Rounding	See BR-GLOBAL-09
Non-negative values	CHECK constraints. CHANGED V1.0 said <code>unsigned</code> ; MySQL 8 deprecated unsigned decimals and Laravel 13 removed <code>unsignedDecimal()</code> .

6.4 Warehouse access

No user-to-warehouse table. A sales employee sells from whichever warehouse holds stock. `warehouses.supervisor_id` is informational, not an access restriction. The warehouse list on the sales screen belongs to `sales.create`, not `warehouses.view`.

6.5 Document numbering **UPDATED**

References `INV`, `PO`, `ADJ`, `TR`, `RET` are issued per year from `document_sequences`. `DocumentSequence::next()` performs one atomic statement:

```
INSERT INTO document_sequences (document_type, year, last_number) VALUES (?, ?, LAST_INSERT_ID(1))
ON DUPLICATE KEY UPDATE last_number = LAST_INSERT_ID(last_number + 1)
```

Called inside the document's transaction, a rollback releases the number. Verified with two concurrent processes issuing 80 numbers: no duplicates, no gaps. Format `PREFIX-YYYY-NNNNN`; the invoice prefix comes from settings.

6.6 Categories, customers, deletion

- Categories are flat.
- `sales.customer_id` is nullable (walk-in sales). A customer's phone is unique when provided.
- Deactivation uses `status` (`active` / `inactive`). SoftDeletes are rejected: they hide rows silently, and a hidden product may still have stock on a shelf.

6.7 Authentication and sessions **V1.1**

- "Remember me" is removed.** Terminals are shared; a remembered login would record one employee's sales under another's name. The `remember_token` column does not exist and the User model disables the feature.
- The login session ends when the browser closes** (`SESSION_EXPIRE_ON_CLOSE=true`), with 120 minutes idle lifetime.
- Invitations are valid for 24 hours**, single use, stored as a SHA-256 hash.
- The first admin** is created by an interactive seeder that asks for the password with hidden input (at least 12 characters, letters and numbers). The password is never written to a file.
- Pending: force a password change on first login when the deploying developer set the admin password (Appendix E).

6.8 Time zone **V1.1**

Application time zone `Africa/Tripoli`; MySQL session time zone `+02:00` (a numeric offset works without MySQL time-zone tables, and Libya has had no daylight saving since 2013). This keeps database defaults and application timestamps identical and prevents invoices near midnight on 31 December from taking the wrong year.

6.9 Language

Arabic, right-to-left interface with Arabic data entry. SKUs, document references and numerals stay Latin, rendered with directional isolation and tabular figures.

7. Functional Requirements

7.1 Employee onboarding

admin creates account (status pending, no password) → invitation issued (24 h, hashed token)
→ employee opens link → sets own password → status active

The admin never sets an employee's password. Employees are disabled, never deleted.

7.2 Roles and permissions

51 system permissions in 11 modules (Appendix A). Roles are configurable; a role in use or a system role cannot be deleted. Re-running the seeders never overwrites role permissions an admin has customised; only the admin role is re-synchronised to hold every permission.

7.3 Products and units

SKU required and unique. Creating a product never creates stock. Inactive products and inactive units cannot be sold. Price changes never alter historical lines. Stock level: **out** at 0 (red), **low** at or below `reorder_level` (yellow), otherwise **available**.

7.4 Create sale

authenticated employee + idempotency key from the form	
→ resolve the employee's OPEN session (never trust a client session id)	[service]
→ validate warehouse, products and units active, quantities > 0	[service]
→ BEGIN	
lock warehouse_stocks rows in a consistent order	[service]
verify stock in base units	[service + CHECK]
compute discount distribution, tax, totals, net prices	[service + CHECKs]
issue invoice number	DocumentSequence::next
insert sale and lines (session must belong to the employee)	[composite FK]
decrement stock, insert SALE movements with negative quantity	[CHECKs]
COMMIT	
→ invoice	

7.5 Sales session

At most one open session per employee (database). Every sale and return attaches to the employee's own open session. Closing records expected cash, actual cash, the difference and notes; a shortage never blocks closing; the difference must equal actual minus expected. Closed sessions are never reopened.

7.6 Purchasing

Statuses `draft` → `confirmed` → `received` or `cancelled`. **V1.1** Confirmation and cancellation record who and when, and the database rejects any status without its matching who/when columns. Receiving selects the warehouse, increments stock with `PURCHASE_RECEIPT` movements and can happen once per order in V1.

7.7 Stock adjustment

Reason is mandatory and cannot be blank. Each line stores previous, new and difference in base units; the difference must be the exact subtraction and cannot be zero. The resulting stock cannot be negative.

7.8 Stock transfer

Source and destination must differ (CHECK). Each line produces `TRANSFER_OUT` (negative) and `TRANSFER_IN` (positive) movements in one transaction.

7.9 Returns and refunds

- Only **cash** sales can be returned, enforced in the database.
- A return line must belong to the invoice being returned (two composite foreign keys sharing `sale_id`).
- `returned_quantity` on the sale line can never exceed the sold quantity (CHECK), even under concurrent returns.
- Refund = one per return, cash, and its amount must equal the return total exactly.
- The return total is computed from net prices. On the final return of a line, the refund uses the remaining line amount so rounding never leaves the customer short. **SERVICE**
- Reason mandatory and non-blank. The original sale is never modified.

7.10 Stock movements

Type (stored value)	Sign (CHECK)	Reference alias
purchase_receipt	positive	purchase_receipt
sale	negative	sale
return	positive	return
adjustment	non-zero	stock_adjustment
transfer_out	negative	stock_transfer
transfer_in	positive	stock_transfer

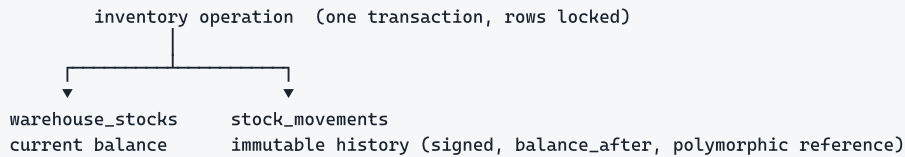
7.11 Reporting and settings

Reports are read-only, permission-filtered, and use `sale_items.unit_cost` captured at sale time for profit. Seeded settings: `company_name`, `company_phone`, `company_address`, `company_logo`, `currency` (LYD), `tax_rate` (0.00), `invoice_prefix` (INV). Critical settings (currency, tax rate, invoice prefix) are audited on change.

8. Data Model

31 application tables, 53 foreign keys, 38 CHECK constraints, plus 8 Laravel system tables (migrations, sessions, cache, cache_locks, jobs, job_batches, failed_jobs, password_reset_tokens).

8.1 Balance and history



8.2 The composite-key pattern V1.1

A foreign key over two columns proves that a row belongs to the right parent, not merely that both ids exist. The parent carries a unique index on the pair.

Guarantee	Mechanism
A unit belongs to the line's product	<code>(product_unit_id, product_id) → product_units(id, product_id)</code>
A sale is in the seller's own session	<code>sales(sales_session_id, user_id) → sales_sessions(id, user_id)</code>
A return is in the returner's own session	<code>returns(sales_session_id, user_id) → sales_sessions(id, user_id)</code>
Only cash sales are returned	<code>returns(sale_id, sale_payment_method) → sales(id, payment_method)</code> plus <code>CHECK (sale_payment_method = 'cash')</code>
A return line belongs to the returned invoice	<code>return_items(return_id, sale_id) → returns(id, sale_id)</code> and <code>return_items(sale_item_id, sale_id) → sale_items(id, sale_id)</code>
Refund equals return total	<code>refunds(return_id, amount) → returns(id, total_amount)</code>

8.3 Invariants enforced by the database

Invariant	Mechanism
Stock never negative	CHECK on balances, balance_after and adjustment quantities
Movement sign matches its type	<code>stock_movements_direction_check</code>
One open session per employee	Generated <code>active_key</code> + unique index
Exactly one base unit per product, factor 1	Generated <code>base_key</code> + unique index + CHECK
Base quantity matches unit factor	CHECK with <code>TRUNCATE</code>
Invoice discount, tax, total and line amounts are arithmetically correct	Five CHECKs with <code>ROUND</code>
Nothing is returned more than it was sold	<code>0 ≤ returned_quantity ≤ quantity</code>
Session and purchase status agree with their dates and amounts	Status consistency CHECKs
One receipt per order; one refund per return	Unique indexes
Transfer source ≠ destination; reasons are not blank	CHECKs
No duplicate submissions; unique document references	Unique idempotency keys and reference numbers
No orphaned history	<code>RESTRICT</code> foreign keys; only true child lines cascade from their header

8.4 Rules that services must enforce SERVICE

These need other rows, sums or live state that a CHECK cannot read. They were identified explicitly, one of them by testing that the database accepts it.

#	Rule
1	Sales and returns only in an open session (verified: the database accepts a sale on a closed session).
2	Invoice subtotal equals the sum of its lines; return total equals the sum of its lines.
3	A return line's unit price equals the sale line's net unit price; its base quantity uses the sale line's factor.

4	Sufficient stock checked under row lock before decrementing (the CHECK is the last line of defence).
5	Products, units, warehouses, suppliers and customers are active when used.
6	A line's frozen <code>unit_factor</code> equals the unit's factor at the time of the transaction.
7	Stock movements are created only by the inventory service; the polymorphic reference cannot be a foreign key.
8	Expected cash at closing = opening float + cash sales – cash refunds for that session.

8.5 Schema reference (generated from the live database)

Every table below is exactly as deployed. "Notes" explain intent; constraints are listed under each table.

Identity and governance

1 roles

Configurable roles. One role per user.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(50)	NOT NULL	
slug	varchar(50)	NOT NULL	
description	varchar(255)	NULL	
is_system	tinyint(1)	NOT NULL, default 0	Only the admin role is protected from deletion
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(name)		
UNIQUE	(slug)		

2 permissions

System-defined capabilities, seeded; never edited through the interface. No timestamps.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
module	varchar(50)	NOT NULL	Groups permissions in the role editor (renamed from "group", a reserved word)
description	varchar(255)	NULL	
UNIQUE	(name)		

3 permission_role

Pivot between roles and permissions. Composite primary key prevents duplicate grants.

Column	Type	Null / default	Notes
role_id	bigint unsigned	NOT NULL, PK	
permission_id	bigint unsigned	NOT NULL, PK	
PRIMARY KEY	(role_id, permission_id)		
FOREIGN KEY	(permission_id) → permissions(id) ON DELETE CASCADE		
FOREIGN KEY	(role_id) → roles(id) ON DELETE CASCADE		

4 users

Employees. No email_verified_at (accepting the invitation proves the address) and no remember_token ("remember me" disabled).

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
role_id	bigint unsigned	NOT NULL	Exactly one role; RESTRICT blocks deleting a role that is in use
name	varchar(100)	NOT NULL	

email	varchar(150)	NOT NULL	
phone	varchar(20)	NULL	
password	varchar(255)	NULL	NULL until the invitation is accepted; bcrypt via the hashed cast
status	enum('pending','active','disabled')	NOT NULL, default pending	Login is refused unless active
last_login_at	timestamp	NULL	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(email)		
FOREIGN KEY	(role_id) → roles(id) ON DELETE RESTRICT		

5 user_invitations

Temporary, single-use onboarding links. Valid for 24 hours.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
user_id	bigint unsigned	NOT NULL	
token_hash	char(64)	NOT NULL	SHA-256 of the token; the raw token is only ever in the e-mailed link
expires_at	timestamp	NOT NULL	
accepted_at	timestamp	NULL	Non-null means used (single use)
invited_by	bigint unsigned	NOT NULL	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(token_hash)		
FOREIGN KEY	(invited_by) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(user_id) → users(id) ON DELETE CASCADE		

6 audit_logs

Immutable record of sensitive actions. No updated_at.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
user_id	bigint unsigned	NULL	
action	varchar(50)	NOT NULL	
auditable_type	varchar(255)	NOT NULL	Short morph alias (e.g. "user", "sale"), never a class name
auditable_id	bigint unsigned	NOT NULL	
old_values	json	NULL	
new_values	json	NULL	
reason	varchar(255)	NULL	
ip_address	varchar(45)	NULL	
terminal_name	varchar(50)	NULL	Which point of sale performed the action
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		

7 settings

Key/value business settings. Only value is editable from the interface.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	Renamed from "key" (reserved word)
value	text	NULL	

type	enum('string','integer','decimal','boolean','json')	NOT NULL, default string	How the text value is cast; decimals stay strings, never float
section	varchar(50)	NOT NULL	Renamed from "group" (reserved word)
is_critical	tinyint(1)	NOT NULL, default 0	Changes are always audited
updated_by	bigint unsigned	NULL	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(name)		
FOREIGN KEY	(updated_by) → users(id) ON DELETE RESTRICT		

Master data

8 categories

Flat categories (no nesting).

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
description	varchar(255)	NULL	
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(name)		

9 brands

Product brands.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(name)		

10 warehouses

Physical locations across cities.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
code	varchar(20)	NOT NULL	
city	varchar(50)	NULL	
address	varchar(255)	NULL	
phone	varchar(20)	NULL	
supervisor_id	bigint unsigned	NULL	Informational only; not an access restriction
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(code)		
FOREIGN KEY	(supervisor_id) → users(id) ON DELETE RESTRICT		

11 customers

Business entity only; customers never log in.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
phone	varchar(20)	NULL	Unique when provided; many walk-in customers may have none
email	varchar(150)	NULL	
address	varchar(255)	NULL	
notes	text	NULL	
status	enum('active','inactive')	NOT NULL, default active	
created_by	bigint unsigned	NULL	Employee who registered the customer
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(phone)		
FOREIGN KEY	(created_by) → users(id) ON DELETE RESTRICT		

12 suppliers

Suppliers of purchase orders.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
name	varchar(100)	NOT NULL	
contact_person	varchar(100)	NULL	
phone	varchar(20)	NULL	
email	varchar(150)	NULL	
address	varchar(255)	NULL	
notes	text	NULL	
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	

13 products

One row per sellable SKU. Prices live on product_units. Creating a product never creates stock.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
sku	varchar(50)	NOT NULL	
name	varchar(150)	NOT NULL	
category_id	bigint unsigned	NULL	
brand_id	bigint unsigned	NULL	
description	text	NULL	
reorder_level	decimal(12,3)	NOT NULL, default 0.000	In base units; at or below it the stock row is shown yellow, at zero red
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(sku)		
FOREIGN KEY	(brand_id) → brands(id) ON DELETE RESTRICT		
FOREIGN KEY	(category_id) → categories(id) ON DELETE RESTRICT		
CHECK	products_reorder_level_check: (reorder_level >= 0)		

14 product_units

Units of measure per product (piece, dozen, carton) with explicit per-unit prices. Stock is always kept in the base unit.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
product_id	bigint unsigned	NOT NULL	
name	varchar(30)	NOT NULL	
factor	decimal(12,4)	NOT NULL	How many base units this unit contains
is_base	tinyint(1)	NOT NULL, default 0	
barcode	varchar(50)	NULL	
purchase_price	decimal(15,3)	NOT NULL, default 0.000	Explicit per unit, never derived by multiplication (wholesale is cheaper)
selling_price	decimal(15,3)	NOT NULL, default 0.000	
status	enum('active','inactive')	NOT NULL, default active	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
base_key	bigint unsigned stored AS (if(is_base,product_id,NULL))	NULL	Stored generated column; its unique index allows exactly one base unit per product
UNIQUE	(barcode)		
UNIQUE	(base_key)		
UNIQUE	(id,product_id)		
UNIQUE	(product_id,name)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
CHECK	product_units_base_factor_check: ((is_base = 0) or (factor = 1))		
CHECK	product_units_factor_check: (factor > 0)		
CHECK	product_units_prices_check: ((purchase_price >= 0) and (selling_price >= 0))		

Inventory core

15 warehouse_stocks

Current balance per product per warehouse, in base units. Changed only by inventory services, under row locks.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
warehouse_id	bigint unsigned	NOT NULL	
product_id	bigint unsigned	NOT NULL	
quantity	decimal(12,3)	NOT NULL, default 0.000	CHECK guarantees it can never be negative
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(warehouse_id,product_id)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY	(warehouse_id) → warehouses(id) ON DELETE RESTRICT		
CHECK	warehouse_stocks_quantity_check: (quantity >= 0)		

16 stock_movements

Immutable history explaining every change to warehouse_stocks. No updated_at.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
product_id	bigint unsigned	NOT NULL	
warehouse_id	bigint unsigned	NOT NULL	
movement_type	enum('purchase_receipt','sale','return','adjustment','transfer_out','transfer_in')	NOT NULL	

quantity	decimal(12,3)	NOT NULL	Signed; the direction CHECK ties the sign to the movement type
balance_after	decimal(12,3)	NOT NULL	Balance immediately after this movement; allows reconciling history with the current balance
reference_type	varchar(255)	NOT NULL	Morph alias of the source document: purchase_receipt, sale, return, stock_adjustment, stock_transfer
reference_id	bigint unsigned	NOT NULL	
user_id	bigint unsigned	NOT NULL	
reason	varchar(255)	NULL	
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(warehouse_id) → warehouses(id) ON DELETE RESTRICT		
CHECK	stock_movements_balance_check: (balance_after >= 0)		
CHECK	stock_movements_direction_check: (((movement_type in ('purchase_receipt','return','transfer_in')) and (quantity > 0)) or ((movement_type in ('sale','transfer_out')) and (quantity < 0)) or ((movement_type = 'adjustment') and (quantity <> 0)))		

17 document_sequences

Counters for human-facing references (INV, PO, ADJ, TR, RET) per year. Issued atomically by DocumentSequence::next().

Column	Type	Null / default	Notes
document_type	varchar(20)	NOT NULL, PK	
year	smallint unsigned	NOT NULL, PK	
last_number	int unsigned	NOT NULL, default 0	
PRIMARY KEY	(document_type, year)		

Purchasing

18 purchases

Purchase orders. Creating or confirming a purchase never changes stock.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
reference_number	varchar(30)	NOT NULL	
supplier_id	bigint unsigned	NOT NULL	
status	enum('draft','confirmed','received','cancelled')	NOT NULL, default draft	draft → confirmed → received, or cancelled; consistency CHECK ties status to its who/when columns
subtotal	decimal(15,3)	NOT NULL, default 0.000	
total	decimal(15,3)	NOT NULL, default 0.000	
notes	text	NULL	
created_by	bigint unsigned	NOT NULL	
confirmed_by	bigint unsigned	NULL	
confirmed_at	timestamp	NULL	

cancelled_by	bigint unsigned	NULL	Added in v1.1 for accountability
cancelled_at	timestamp	NULL	
created_at	timestamp	NULL	
updated_at	timestamp	NULL	
UNIQUE	(reference_number)		
FOREIGN KEY	(cancelled_by) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(confirmed_by) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(created_by) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(supplier_id) → suppliers(id) ON DELETE RESTRICT		
CHECK	purchases_amounts_check: ((subtotal >= 0) and (total >= 0))		
CHECK	purchases_status_check: (((status = 'draft') and (confirmed_at is null) and (cancelled_at is null)) or ((status in ('confirmed','received')) and (confirmed_at is not null) and (confirmed_by is not null) and (cancelled_at is null)) or ((status = 'cancelled') and (cancelled_at is not null) and (cancelled_by is not null)))		

19 purchase_items

Ordered lines with frozen unit factor and cost.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
purchase_id	bigint unsigned	NOT NULL	
product_id	bigint unsigned	NOT NULL	
product_unit_id	bigint unsigned	NOT NULL	Composite FK (product_unit_id, product_id) guarantees the unit belongs to the product
unit_factor	decimal(12,4)	NOT NULL	
quantity	decimal(12,3)	NOT NULL	
base_quantity	decimal(12,3)	NOT NULL	Must equal TRUNCATE(quantity × unit_factor, 3), identical to PHP bcmul
unit_cost	decimal(15,3)	NOT NULL	
line_total	decimal(15,3)	NOT NULL	
UNIQUE	(purchase_id,product_unit_id)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY (composite)	(product_unit_id,product_id) → product_units(id,product_id) ON DELETE RESTRICT		
FOREIGN KEY	(purchase_id) → purchases(id) ON DELETE CASCADE		
CHECK	purchase_items_amounts_check: ((unit_cost >= 0) and (line_total >= 0))		
CHECK	purchase_items_base_quantity_check: (base_quantity = truncate((quantity * unit_factor),3))		
CHECK	purchase_items_quantity_check: ((quantity > 0) and (unit_factor > 0))		

20 purchase_receipts

The physical receiving event. Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
purchase_id	bigint unsigned	NOT NULL	UNIQUE: one receipt per order in V1; dropping this index enables partial receiving in V2
warehouse_id	bigint unsigned	NOT NULL	
received_by	bigint unsigned	NOT NULL	
notes	text	NULL	
received_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	Event time; the only timestamp
UNIQUE	(purchase_id)		
FOREIGN KEY	(purchase_id) → purchases(id) ON DELETE RESTRICT		
FOREIGN KEY	(received_by) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(warehouse_id) → warehouses(id) ON DELETE RESTRICT		

21 purchase_receipt_items

What actually arrived, per order line. Product and unit are derived from the order line (3NF). Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
purchase_receipt_id	bigint unsigned	NOT NULL	
purchase_item_id	bigint unsigned	NOT NULL	
quantity	decimal(12,3)	NOT NULL	
base_quantity	decimal(12,3)	NOT NULL	
UNIQUE	(purchase_receipt_id,purchase_item_id)		
FOREIGN KEY	(purchase_item_id) → purchase_items(id) ON DELETE RESTRICT		
FOREIGN KEY	(purchase_receipt_id) → purchase_receipts(id) ON DELETE CASCADE		
CHECK	purchase_receipt_items_quantity_check: ((quantity > 0) and (base_quantity > 0))		

Inventory operations

22 stock_adjustments

Corrections after a physical count. Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
reference_number	varchar(30)	NOT NULL	
idempotency_key	char(36)	NOT NULL	Rejects a double-submitted form
warehouse_id	bigint unsigned	NOT NULL	
user_id	bigint unsigned	NOT NULL	
reason	varchar(255)	NOT NULL	Mandatory and cannot be blank spaces
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
UNIQUE	(idempotency_key)		
UNIQUE	(reference_number)		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(warehouse_id) → warehouses(id) ON DELETE RESTRICT		
CHECK	stock_adjustments_reason_check: (char_length(trim(reason)) > 0)		

23 stock_adjustment_items

Balances before and after, always in base units (no unit columns). Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
stock_adjustment_id	bigint unsigned	NOT NULL	
product_id	bigint unsigned	NOT NULL	
previous_quantity	decimal(12,3)	NOT NULL	
new_quantity	decimal(12,3)	NOT NULL	
difference	decimal(12,3)	NOT NULL	Must equal new_quantity – previous_quantity and cannot be zero
UNIQUE	(stock_adjustment_id,product_id)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY	(stock_adjustment_id) → stock_adjustments(id) ON DELETE CASCADE		
CHECK	stock_adjustment_items_difference_check: ((difference = (new_quantity - previous_quantity)) and (difference <> 0))		
CHECK	stock_adjustment_items_quantities_check: ((previous_quantity >= 0) and (new_quantity >= 0))		

24 stock_transfers

Moves stock between two different warehouses. Immutable.

Column	Type	Null / default	Notes
--------	------	----------------	-------

id	bigint unsigned	NOT NULL, auto increment, PK	
reference_number	varchar(30)	NOT NULL	
idempotency_key	char(36)	NOT NULL	Rejects a double-submitted form
from_warehouse_id	bigint unsigned	NOT NULL	
to_warehouse_id	bigint unsigned	NOT NULL	
user_id	bigint unsigned	NOT NULL	
notes	text	NULL	
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
UNIQUE	(idempotency_key)		
UNIQUE	(reference_number)		
FOREIGN KEY	(from_warehouse_id) → warehouses(id) ON DELETE RESTRICT		
FOREIGN KEY	(to_warehouse_id) → warehouses(id) ON DELETE RESTRICT		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
CHECK	stock_transfers_warehouses_check: (from_warehouse_id <> to_warehouse_id)		

25 stock_transfer_items

Transferred packages, keeping the unit so the receiving warehouse can match what arrived. Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
stock_transfer_id	bigint unsigned	NOT NULL	
product_id	bigint unsigned	NOT NULL	
product_unit_id	bigint unsigned	NOT NULL	Composite FK guarantees the unit belongs to the product
unit_factor	decimal(12,4)	NOT NULL	
quantity	decimal(12,3)	NOT NULL	
base_quantity	decimal(12,3)	NOT NULL	
UNIQUE	(stock_transfer_id,product_id)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY (composite)	(product_unit_id,product_id) → product_units(id,product_id) ON DELETE RESTRICT		
FOREIGN KEY	(stock_transfer_id) → stock_transfers(id) ON DELETE CASCADE		
CHECK	stock_transfer_items_base_quantity_check: (base_quantity = truncate((quantity * unit_factor),3))		
CHECK	stock_transfer_items_quantity_check: ((quantity > 0) and (unit_factor > 0))		

Selling

26 sales_sessions

An employee's selling shift. Not tied to a warehouse. Holds the cash reconciliation.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
user_id	bigint unsigned	NOT NULL	
status	enum('open','closed')	NOT NULL, default open	
opening_float	decimal(15,3)	NOT NULL, default 0.000	Cash counted into the drawer at session start
expected_cash	decimal(15,3)	NULL	
actual_cash	decimal(15,3)	NULL	
difference	decimal(15,3)	NULL	Must equal actual_cash – expected_cash; a shortage never blocks closing
closing_notes	text	NULL	
opened_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
closed_at	timestamp	NULL	

active_key	bigint unsigned stored AS (if((status = 'open'),user_id,NULL))	NULL	Stored generated column; unique index allows one open session per employee
UNIQUE	(active_key)		
UNIQUE	(id,user_id)		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
CHECK	sales_sessions_cash_check: ((opening_float >= 0) and ((actual_cash is null) or (actual_cash >= 0)))		
CHECK	sales_sessions_status_check: (((status = 'open') and (closed_at is null) and (expected_cash is null) and (actual_cash is null) and (difference is null)) or ((status = 'closed') and (closed_at is not null) and (closed_at >= opened_at) and (expected_cash is not null) and (actual_cash is not null) and (difference = (actual_cash - expected_cash))))		

27 sales

Invoice header. Immutable once written.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
invoice_number	varchar(30)	NOT NULL	
idempotency_key	char(36)	NOT NULL	Rejects a double-submitted sale
sales_session_id	bigint unsigned	NOT NULL	Composite FK (sales_session_id, user_id): the session must belong to the same employee
user_id	bigint unsigned	NOT NULL	
customer_id	bigint unsigned	NULL	
warehouse_id	bigint unsigned	NOT NULL	
payment_method	enum('cash', 'card', 'transfer')	NOT NULL	
subtotal	decimal(15,3)	NOT NULL	
discount_percent	decimal(5,2)	NOT NULL, default 0.00	
discount_amount	decimal(15,3)	NOT NULL, default 0.000	Must equal ROUND(subtotal × discount_percent / 100, 3)
tax_rate	decimal(5,2)	NOT NULL, default 0.00	Rate in force at the time of sale, frozen
tax_amount	decimal(15,3)	NOT NULL, default 0.000	Must equal ROUND((subtotal – discount_amount) × tax_rate / 100, 3); tax applies after discount
total	decimal(15,3)	NOT NULL	Must equal subtotal – discount_amount + tax_amount
notes	varchar(255)	NULL	
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
UNIQUE	(id,payment_method)		
UNIQUE	(idempotency_key)		
UNIQUE	(invoice_number)		
FOREIGN KEY	(customer_id) → customers(id) ON DELETE RESTRICT		
FOREIGN KEY (composite)	(sales_session_id,user_id) → sales_sessions(id,user_id) ON DELETE RESTRICT		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
FOREIGN KEY	(warehouse_id) → warehouses(id) ON DELETE RESTRICT		
CHECK	sales_amounts_check: ((subtotal >= 0) and (discount_amount >= 0) and (tax_amount >= 0) and (total >= 0))		
CHECK	sales_discount_check: (discount_amount = round(((subtotal * discount_percent) / 100),3))		
CHECK	sales_rates_check: ((discount_percent between 0 and 100) and (tax_rate between 0 and 100))		
CHECK	sales_tax_check: (tax_amount = round((((subtotal – discount_amount) * tax_rate) / 100),3))		
CHECK	sales_total_check: (total = ((subtotal – discount_amount) + tax_amount))		

28 sale_items

Invoice lines. Final once written, except the returned-quantity counter.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
sale_id	bigint unsigned	NOT NULL	

product_id	bigint unsigned	NOT NULL	
product_unit_id	bigint unsigned	NOT NULL	
unit_factor	decimal(12,4)	NOT NULL	
quantity	decimal(12,3)	NOT NULL	
base_quantity	decimal(12,3)	NOT NULL	
unit_price	decimal(15,3)	NOT NULL	
unit_cost	decimal(15,3)	NOT NULL	Cost per base unit at the time of sale, for profit reporting
discount_share	decimal(15,3)	NOT NULL, default 0.000	This line's share of the invoice discount
line_total	decimal(15,3)	NOT NULL	Must equal ROUND(quantity × unit_price, 3) – discount_share
net_unit_price	decimal(15,3)	NOT NULL	Must equal ROUND(line_total / quantity, 3); the basis for refunds
returned_quantity	decimal(12,3)	NOT NULL, default 0.000	CHECK keeps it between 0 and quantity: nothing can be returned twice
UNIQUE	(id,sale_id)		
UNIQUE	(sale_id,product_unit_id)		
FOREIGN KEY	(product_id) → products(id) ON DELETE RESTRICT		
FOREIGN KEY (composite)	(product_unit_id,product_id) → product_units(id,product_id) ON DELETE RESTRICT		
FOREIGN KEY	(sale_id) → sales(id) ON DELETE CASCADE		
CHECK	sale_items_amounts_check: ((unit_price >= 0) and (unit_cost >= 0) and (discount_share >= 0) and (line_total >= 0))		
CHECK	sale_items_base_quantity_check: (base_quantity = truncate((quantity * unit_factor),3))		
CHECK	sale_items_line_total_check: (line_total = (round((quantity * unit_price),3) – discount_share))		
CHECK	sale_items_net_price_check: (net_unit_price = round((line_total / quantity),3))		
CHECK	sale_items_quantity_check: ((quantity > 0) and (unit_factor > 0))		
CHECK	sale_items_returned_check: ((returned_quantity >= 0) and (returned_quantity <= quantity))		

Returns and refunds

29 returns

Customer return against one cash sale. Immutable. Stock returns to the sale's warehouse, so no warehouse column.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
reference_number	varchar(30)	NOT NULL	
idempotency_key	char(36)	NOT NULL	
sale_id	bigint unsigned	NOT NULL	
sale_payment_method	enum('cash','card','transfer')	NOT NULL, default cash	Integrity column: composite FK copies the sale's payment method and a CHECK requires cash
sales_session_id	bigint unsigned	NOT NULL	Composite FK: the session must belong to the same employee
user_id	bigint unsigned	NOT NULL	
reason	varchar(255)	NOT NULL	Mandatory and cannot be blank spaces
total_amount	decimal(15,3)	NOT NULL	
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
UNIQUE	(id,sale_id)		
UNIQUE	(id,total_amount)		
UNIQUE	(idempotency_key)		
UNIQUE	(reference_number)		
FOREIGN KEY (composite)	(sale_id,sale_payment_method) → sales(id,payment_method) ON DELETE RESTRICT		
FOREIGN KEY (composite)	(sales_session_id,user_id) → sales_sessions(id,user_id) ON DELETE RESTRICT		
FOREIGN KEY	(user_id) → users(id) ON DELETE RESTRICT		
CHECK	returns_amount_check: (total_amount > 0)		
CHECK	returns_cash_only_check: (sale_payment_method = 'cash')		
CHECK	returns_reason_check: (char_length(trim(reason)) > 0)		

30 return_items

Returned lines, in the unit that was sold. Product and unit are derived from the sale line. Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
return_id	bigint unsigned	NOT NULL	
sale_id	bigint unsigned	NOT NULL	Shared by two composite FKs so the return and the sale line must belong to the same invoice
sale_item_id	bigint unsigned	NOT NULL	
quantity	decimal(12,3)	NOT NULL	
base_quantity	decimal(12,3)	NOT NULL	
unit_price	decimal(15,3)	NOT NULL	The sale line's net (post-discount) price
line_total	decimal(15,3)	NOT NULL	
UNIQUE	(return_id,sale_item_id)		
FOREIGN KEY (composite)	(return_id,sale_id) → returns(id,sale_id) ON DELETE CASCADE		
FOREIGN KEY (composite)	(sale_item_id,sale_id) → sale_items(id,sale_id) ON DELETE RESTRICT		
CHECK	return_items_amounts_check: ((unit_price >= 0) and (line_total >= 0))		
CHECK	return_items_quantity_check: ((quantity > 0) and (base_quantity > 0))		

31 refunds

Cash handed back for a return. Exactly one per return. Employee and session are derived from the return. Immutable.

Column	Type	Null / default	Notes
id	bigint unsigned	NOT NULL, auto increment, PK	
return_id	bigint unsigned	NOT NULL	UNIQUE: one refund per return
amount	decimal(15,3)	NOT NULL	Composite FK (return_id, amount) → returns(id, total_amount): must equal the return total exactly
method	enum('cash')	NOT NULL, default cash	
created_at	timestamp	NOT NULL, default CURRENT_TIMESTAMP	
UNIQUE	(return_id)		
FOREIGN KEY (composite)	(return_id,amount) → returns(id,total_amount) ON DELETE RESTRICT		
CHECK	refunds_amount_check: (amount > 0)		

9. Technical Architecture and Implementation Status

9.1 Stack

Layer	Choice
Framework	Laravel 13.31 (installed packages require PHP ≥ 8.4.1)
Presentation	Blade, Tailwind CSS 4.3, Alpine.js 3.17 with the collapse plugin, Vite 8; Arabic RTL; Cairo font self-hosted
Database	MySQL 8.4.3 — needed for enforced CHECK constraints, indexed stored generated columns and composite foreign keys
Database user	Dedicated least-privilege user limited to the <code>mizan</code> schema; never <code>root</code>
Business logic	Service classes own transactions and locking; controllers stay thin

9.2 Model conventions V1.1

- **30 models, 9 enums** (`UserStatus`, `SettingType`, `ActiveStatus`, `StockMovementType`, `StockLevel`, `PurchaseStatus`, `SessionStatus`, `PaymentMethod`, `DocumentType`).
- **Mass assignment:** sensitive columns (`role`, `status`, `password`, `is_system`, `created_by`, `is_base`) are never fillable. Transaction and inventory models are fully guarded and written only by services.
- **Immutability:** an `Immutable` trait blocks update and delete on historical models; `SaleItem` allows only `returned_quantity`.
- **Morph map (enforced):** polymorphic columns store short stable aliases (`user`, `role`, `setting`, `category`, `brand`, `warehouse`, `customer`, `supplier`, `product`, `product_unit`, `purchase`, `purchase_receipt`, `stock_adjustment`, `stock_transfer`, `sales_session`, `sale`, `return`, `refund`), never class names, so renaming a class cannot break history.
- **Strict mode** outside production: silently discarded attributes, lazy loading and missing attributes raise errors.
- The return model is `SaleReturn` (table `returns`) because `return` is reserved in PHP. Reserved SQL words were avoided as column names.
- Index names are kept within MySQL's 64-character identifier limit.

9.3 Deployment and environments

Warehouses are in different cities, so V1 is one internet-hosted web application with a single shared database. Per-terminal installations and branch-local servers with synchronisation are rejected. Development uses Laragon's MySQL 8.4.3 and Herd's PHP 8.4 from PowerShell; Laragon's own terminal ships PHP 8.3 and cannot run the project.

Schema changes on a database that holds data use `php artisan migrate`, never `migrate:--fresh`. Migrations are grouped in batches so `migrate:rollback` undoes only the last one.

9.4 Implementation status (16 Sep 2026)

Area	Status	Verification
Project, Arabic RTL layout, dashboard mock-up	Done	Rendered in the browser
Schema: 31 tables in 4 migration batches	Done	Every constraint attacked on a throwaway database, then re-tested on the real one; every rollback tested
Models, enums, relationships	Done	110 automated checks inside rolled-back transactions
Seeders: permissions, roles, settings, first admin	Done	Re-run safety confirmed
Document numbering under concurrency	Done	2 processes, 80 numbers, no duplicates or gaps
Authentication, services, screens, reports	Not started	—

10. Security Requirements

Area	Requirement
Passwords	bcrypt via the <code>hashed</code> cast; admin password at least 12 characters with letters and numbers; never logged, never typed into tinker (its history is saved to disk).
Invitation tokens	Random, sent once, stored as SHA-256, compared in constant time, 24 h, single use.
Shared terminals	No "remember me"; session ends when the browser closes.
Authorisation	Server-side on every operation; the active session is derived from the authenticated user.

Money	Computed server-side; the database re-checks the arithmetic.
Database access	Least-privilege application user. Production: revoke <code>UPDATE</code> and <code>DELETE</code> on <code>audit_logs</code> (and consider <code>stock_movements</code>) from that user.
Secrets	Only in <code>.env</code> , which is excluded from version control. Temporary setup scripts containing credentials were deleted.

11. Out of Scope — V2 Roadmap

Feature	V1 readiness
Partial goods receiving	Drop the unique index on <code>purchase_receipts.purchase_id</code> ; receipt lines already reference order lines
Partial or non-cash refunds	Drop the <code>(return_id, amount)</code> foreign key; extend <code>refunds.method</code>
Returning card or transfer sales	Drop <code>returns_cash_only_check</code>
Returning a different unit than sold	Service-level conversion on return lines
Line-level discounts	Lines already store <code>discount_share</code> and <code>net_unit_price</code>
Product variants, multi-tenancy, customer portal, delivery tracking, payment gateways	Not prepared; require new layers

Appendix A — Permission Catalogue (51)

users (4)	users.view users.create users.update users.disable	inventory (4)	inventory.view inventory.adjust inventory.transfer stock_movements.view
roles (4)	roles.view roles.create roles.update roles.delete	purchases (4)	purchases.view .create .confirm .receive
catalog (6)	products.view products.create products.update products.deactivate categories.manage brands.manage	sales (9)	sessions.start .close .view_all sales.create .view_own .view_all returns.create .view_own .view_all
suppliers (4)	suppliers.view .create .update .deactivate	reports (5)	reports.sales .purchases .inventory .profit .movements
customers (4)	customers.view .create .update .deactivate	governance (3)	settings.view settings.update audit.view
warehouses (4)	warehouses.view .create .update .deactivate		

Appendix B — Role / Permission Matrix

Permission group	Admin (51)	Manager (31)	Sales (11)	Warehouse (9)
Users, roles, settings, audit	Full	—	—	—
Products, categories, brands	Full	Full	View products	View products
Suppliers	Full	Full	—	View
Customers	Full	Full	View, create, update	—
Warehouses	Full	Full	—	View
Inventory view, movements	Full	Both	Inventory only	Both
Adjust, transfer	Full	—	—	Both
Purchases	Full	View, create, confirm	—	View, receive
Sessions	Full	View all	Start, close	—
Sales	Full	View all	Create, view own	—
Returns and refunds	Full	View all	Create, view own	—
Reports	All 5	All 5	—	—

Appendix C — Decision Register

#	Question	Decision
C1–C8	Source-document conflicts	See section 5
D1	Tables missing from the V1.0 ERD list	Added categories, brands, audit_logs, settings, user_invitations, product_units, document_sequences
D2	User-to-warehouse assignment	Rejected; supervisor is informational
D3	Negative stock	Never, by any operation
D4	Invoice identity	Column on <code>sales</code> , issued from the sequence
D5	Money precision	<code>decimal(15,3)</code>
D6	Profit basis	<code>unit_cost</code> per base unit frozen on sale lines
D7	Tax	NOT NULL default 0; rate frozen per sale; applied after discount
D8	Discounts	Invoice-level percentage; refunds at net price
D9	Language	Arabic RTL; SKUs and numerals Latin
D10	Deletion	Status column; no soft deletes
D11	Units of measure	Multi-unit, frozen factors, per-unit prices
D12	Quantity type	<code>decimal(12,3)</code>
D13	Categories	Flat
D14	Walk-in sales	Allowed
D15	Purchase cancellation	Status added, with who and when
D16	Deployment	Single hosted web application
D17	Movement balance	<code>balance_after</code> on every movement
D18	"Remember me"	Removed; session ends when the browser closes
D19	Invitation validity	24 hours
D20	First admin	Interactive seeder, hidden password input, ≥12 chars with letters and numbers
D21	Reserved words	<code>permissions.module</code> , <code>settings.section</code> , <code>settings.name</code>
D22	Time zone	Africa/Tripoli; database session +02:00
D23	Non-negative decimals	CHECK constraints instead of unsigned
D24	Deleting a product that has units	RESTRICT (MySQL forbids CASCADE on the base column of a stored generated column)
D25	Movement type values	Lowercase, matching morph aliases
D26	Movement sign	CHECK ties sign to type
D27	Unit/product consistency	Composite foreign keys
D28	Base quantity rounding	Truncate to 3 decimals
D29	Purchase status integrity	Status consistency CHECK
D30	Receipt lines	Reference order lines, not products
D31	Adjustment lines	Base units only, no unit columns
D32	Duplicate submissions	Idempotency keys on sales, returns, adjustments, transfers
D33	Money rounding	Half away from zero, 3 decimals
D34	Session ownership	Enforced on sales and returns by composite keys
D35	Cash-only returns	Enforced in the database
D36	Return unit	The unit that was sold
D37	Return line ownership	Must belong to the returned invoice
D38	Refund	Equals return total; employee and session derived from the return
D39	Historical records	Immutable at model level

D40	<code>currency_decimals</code> setting	Removed; precision is fixed by the schema
D41	Document numbering	Atomic <code>LAST_INSERT_ID</code> statement
D42	Session close	<code>opened_at</code> / <code>closed_at</code> ; difference = actual – expected
D43	Adjustment semantics	The employee enters the <i>counted</i> quantity, not a delta. <code>previous_quantity</code> is read under lock at save time, never taken from the screen, so a concurrent sale cannot corrupt the result.
D44	Adjustment lines with no difference	Skipped silently (the line CHECK forbids a zero difference, and such a line carries no information). If <i>every</i> line matches, the whole adjustment is refused with a clear message rather than writing an empty document.
D45	Transfer deadlock safety	All balance rows both warehouses will touch are locked up front in one global (<code>warehouse_id</code> , <code>product_id</code>) order, so an A→B and a B→A transfer running at the same instant cannot hold each other's next lock. Verified with two concurrent opposite-direction processes.
D46	Adjustment/transfer visibility	Viewing the logs requires <code>inventory.view</code> ; performing them requires <code>inventory.adjust</code> / <code>inventory.transfer</code> , held only by the warehouse role. Managers review without being able to execute.
D47	Current balances in the entry screens	Known limitation: the adjustment, transfer and point-of-sale forms embed every warehouse×product balance in the page as JSON, so the current quantity appears with no round trip. The payload grows as products × warehouses — negligible at a few hundred products, roughly 1.5 MB at 5,000 products × 10 warehouses. Replace it with an on-demand lookup keyed by the selected warehouse when the catalogue approaches that size. The displayed figure is advisory only; the authoritative balance is always re-read under lock when saving.
D48	Price authority at the till	The till submits units and quantities only; every price is read from the product card server side and a price posted in the form is ignored. The cashier's only lever is the invoice discount percentage, which is stored and auditable. Prices themselves are set in the products screen, which the admin and manager roles hold (<code>products.update</code>).
D49	Spreading the invoice discount	Largest-remainder allocation: each line takes its proportional share truncated to three decimals, then the leftover millimes go to the lines with the largest discarded fraction. Rounding each line independently drifts by a millime or two and breaks <code>sales_discount_check</code> . Verified with a 33.33% discount over lines of 1/2/4.
D50	Resolving the selling session	The service resolves the employee's own open session and locks it inside the sale transaction. A session id is never accepted from the client, and the lock closes the window where a session is closed in another tab while a sale is being posted. The database enforces session ownership by composite key but does not enforce that it is open.
D51	The same unit scanned twice	Merged into one line with the quantities added, because <code>sale_items</code> carries a unique(sale_id, product_unit_id). Rejecting the invoice would be hostile at a till.
D52	Screens two permissions can reach	Composite gate abilities <code>sales.browse</code> and <code>sessions.browse</code> : a seller reaches them through the "own" permission, a manager through the "all" one. Without these a manager, who holds only <code>sales.view_all</code> , would be locked out of the invoices list.
D53	Alpine <code>x-show</code> on the till	Implementation note: two <code>x-show</code> expressions on the point-of-sale screen registered their effect but never re-evaluated, while <code>x-text</code> and <code>:class</code> on the same component reacted normally. Both were switched to a <code>:class="... && 'hidden'"</code> toggle, which is verified working. Prefer that pattern on this screen rather than reintroducing <code>x-show</code> .
D54	Partial and repeated returns	Allowed and free: <code>sale_items.returned_quantity</code> starts at zero and accumulates, so a customer may return 3 pieces today and 4 tomorrow from the same line. The remaining quantity is always re-read under lock at refund time, never taken from the screen, and <code>returned_quantity <= quantity</code> is a database CHECK. Forbidding this would have required extra code, not less.
D55	Which session a refund belongs to	The return attaches to the open session of whoever is serving now, not to the session that made the sale, because the cash physically leaves today's drawer. A refund against an invoice from a closed session therefore shows as a shortfall in today's session.
D56	Refund price	The net (post-discount) unit price frozen on the original sale line. A piece sold at 10 under a 20% invoice discount refunds 8, never 10, so a customer cannot profit by buying at a discount and returning at full price.
D57	Lock ordering inside <code>lockBalances()</code>	Both the warehouse and product id lists are sorted before the <code>INSERT IGNORE</code> that creates any missing balance rows. MySQL takes locks in the order a VALUES list is written, so two callers naming the same warehouses in opposite order deadlocked on that statement before the ordered <code>SELECT ... FOR UPDATE</code> ran. Found by an intermittent failure of the opposite-direction transfer test; it only triggers when both balance rows are missing.

Appendix D — Change Log V1.0 → V1.1

Area	V1.0 said	V1.1 — and why
Users	<code>remember_token</code> , <code>email_verified_at</code>	Both removed. Shared terminals make "remember me" an accountability risk; accepting the invitation already proves the e-mail address.
Sessions (login)	Not specified	Expire when the browser closes; 120-minute idle lifetime.
Invitations	"Temporary"	24 hours.
Permissions	<code>group</code> ; about 50	<code>module</code> (reserved word); exactly 51 in 11 modules.
Settings	<code>key</code> , <code>group</code> ; <code>currency_decimals</code>	<code>name</code> , <code>section</code> ; <code>currency_decimals</code> removed.
Roles	All default roles protected?	Only admin is a system role; seeding never overwrites customised role permissions.
Time zone	Not specified	Africa/Tripoli and +02:00; the default config ignored the environment value and ran in UTC.
Non-negative values	<code>unsigned</code> decimals	CHECK constraints (unsigned decimals deprecated in MySQL, removed from Laravel 13).
product_units FK	CASCADE	RESTRICT: MySQL rejected CASCADE because <code>base_key</code> is generated from <code>product_id</code> (error 1215).
Movement types	Uppercase	Lowercase, consistent with every other stored value and morph alias; sign enforced per type.
Line tables	<code>product_id</code> and <code>product_unit_id</code> independent	Composite foreign key proves the unit belongs to the product.
purchases	Statuses only	<code>cancelled_by</code> / <code>cancelled_at</code> and a status consistency CHECK.
purchase_receipt_items	<code>product_id</code> , <code>product_unit_id</code>	Reference <code>purchase_item_id</code> (3NF; enables partial receiving per line).
purchase_receipts	Timestamps	<code>received_at</code> only; the receipt is an immutable event.
stock_adjustment_items	Unit columns	Removed; balances are in base units.
Adjustments, transfers, sales, returns	Idempotency for sale, receipt, refund	Idempotency keys on sales, returns, adjustments, transfers.
document_sequences	Lock then increment	Single atomic <code>LAST_INSERT_ID</code> statement; concurrency-tested.
sales_sessions	Timestamps	<code>opened_at</code> / <code>closed_at</code> ; status consistency with the difference formula.
sales	No tax rate; timestamps	<code>tax_rate</code> frozen; tax after discount; five arithmetic CHECKS; immutable (created_at only); session must be the seller's.
sale_items	Columns only	Formulas for <code>line_total</code> and <code>net_unit_price</code> ; <code>returned_quantity</code> $\leq$ <code>quantity</code> .
returns	Cash rule in code	Cash-only enforced in the database via <code>sale_payment_method</code> ; no warehouse column.
return_items	<code>product_id</code> , unit	<code>sale_id</code> with two composite keys; product and unit derived from the sale line; quantity in the sold unit.
refunds	<code>user_id</code> , <code>sales_session_id</code>	Removed (derived from the return); amount must equal the return total.
Table count	"26 tables" (V1.0); "46" in discussion	31 application tables + 8 Laravel system tables = 39.
Rounding	Not specified	Half away from zero for money, truncation for base quantities; parity with PHP verified.
Deployment tooling	Not specified	Laragon MySQL 8.4.3 and Herd PHP 8.4 in development; <code>migrate</code> , never <code>fresh</code> , once data exists.

Appendix E — Open Items

#	Item	Proposed resolution
1	The deploying developer knows the first admin's password	Add <code>users.password_changed_at</code> ; force a change at first login. Build with authentication.

2	Admin seeder re-asks every question when only the password is invalid	Ask name and e-mail until valid, then loop on the password alone.
3	E-mail addresses are stored with the case typed	Normalise to lowercase before saving.
4	Audit log protected only at model level	Revoke UPDATE/DELETE on <code>audit_logs</code> for the production database user.
5	Warehouse list on the sales screen	Authorised by <code>sales.create</code> , not <code>warehouses.view</code> .
6	Tinker aliases for new models	Run <code>composer dump-autoload --optimize</code> after adding models.